

Bitonic Sort als Sortieralgorithmus in parallelisierten Anwendungen

Berend Tobias
Angewandte Informatik
Hochschule Offenburg
Offenburg, Deutschland
tberend@stud.hs-offenburg.de

I. EINLEITUNG¹

Sortieralgorithmen spielen seit den Anfängen des Computerzeitalters eine fundamentale Rolle in der Informatik. Da die effiziente Verarbeitung und Strukturierung von Datenmengen essenziell für alle Systeme und Anwendungen sind, werden kontinuierlich neue Algorithmen entwickelt und angepasst. Und auch so ist der Bitonic Sort entstanden.

Der Begriff „bitonisch“ bzw. „bitonic“ beschreibt dabei die Eigenschaft einer Sequenz, die über ein bestimmtes Intervall zunächst monoton steigt und anschließend monoton fällt (oder umgekehrt) [1], [2]. Der Bitonic Sort nutzt hierbei die Eigenheiten solcher Reihen, indem er Daten nacheinander in solche bitonischen Sequenzen überführt und diese anschließend effizient verschmilzt. Ein visuelles Beispiel für ein solches Intervall und dessen Transformation ist in Abb. 1 dargestellt.

Klassische, sequenzielle Sortiervverfahren wie Quick Sort oder Merge Sort erreichen im Durchschnitt eine Zeitkomplexität von $O(n \log(n))$. Bitonic Sort jedoch nur $O(n \log(n)^2)$. Die parallele Implementation hingegen hat eine Komplexität von $O(\log(n)^2)$. Diese benötigt jedoch ein hochparallelisierte Umgebung wie eine GPU [3], [4].

Es stellt sich daher die Frage, ab welcher Datengröße und unter welchen Bedingungen die Nutzung von Bitonic Sort in parallelisierten Anwendungen tatsächlich sinnvoll ist und wann traditionelle, sequenzielle oder parallele Verfahren überlegen bleiben [4].

II. PARALLELE PROZESSE

Parallele Prozesse funktionieren, indem z.B. zwei Prozesse auf zwei unterschiedliche CPU-Kerne aufgeteilt werden, welche unabhängig voneinander Berechnungen durchführen können. Dabei kann die Berechnungszeit halbiert werden (best case), da beide CPU-Kerne nur die Hälfte der Berechnungen abarbeiten müssen.

Dieses Prinzip kann in vielen Bereichen angewandt werden, darunter auch Threads. Threads sind leichtgewichtige Teilprozesse, welche ebenfalls auf mehrere CPUs verteilt werden können und so die Anwendung parallelisieren. Threads eignen sich des Weiteren zur Realisierung des parallelen Bitonic Sorts, da dieser nach dem „divide and conquer“-Prinzip arbeitet, also das Problem rekursiv in kleinere Teilprobleme auflöst und diese Teilprobleme anschließend von anderen Threads lösen lässt.

Auch die GPU nutzt Parallelisierung. Sie besitzt um ein Vielfaches mehr Cores als eine CPU, kann also deutlich mehr Berechnungen gleichzeitig ausführen. Insbesondere NVIDIA

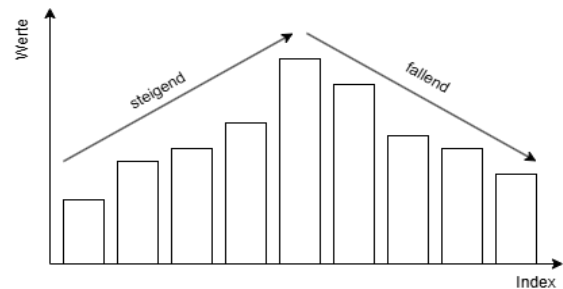


Abbildung 1. Beispiel einer Bitonischen Reihe

Grafikkarten mit der CUDA Technologie und Ray-tracing Cores sind dabei zu nennen [1], [5].

III. FUNKTIONSWEISE BITONIC SORT¹

Wie bereits im vorangegangenen Kapitel II erwähnt, arbeitet der parallele Bitonic Sort nach dem „divide and conquer“-Prinzip. Bei diesem Ansatz wird ein Problem in kleinere Teilprobleme aufgeteilt, bis diese trivial lösbar sind. Im Kontext des Bitonic Sort heißt dies, dass das zu sortierende Array rekursiv in 2 kleinere Teilarrays aufgeteilt, unabhängig voneinander sortiert und anschließend zusammengeführt wird.

Laut [2] hat der Bitonic Sort eine Vorbedingung an die Arraylänge. Diese muss einer Zweierpotenz entsprechen, also:

$$\text{Arraylänge } l = 2^x, x \in \mathbb{N}_0$$

Entspricht die Datenmenge keiner Zweierpotenz, kann mithilfe von Füllwerten das Array künstlich erweitert werden. Da dieses Verfahren jedoch zusätzlichen Speicherplatz und Zeit in Anspruch nimmt, wird es in der Praxis nicht angewandt.

Der Bitonic Sort ist in 2 Teile bzw. Funktionen aufgeteilt. Bitonic Sort und Bitonic Merge. Beide werden in den folgenden Kapiteln besprochen.

A. Bitonic Sort

In Codebeispiel 1 sieht man diesen Algorithmus auf einem Integer Array implementiert in Java. Übergabeparameter der Funktion sind das zu sortierende Array, der Startindex, Endindex und die Richtung, in welche das Array sortiert werden soll. Hierbei steht „true“ für aufsteigend und „false“ für absteigend. Um ein Array dementsprechend absteigend zu sortieren, muss lediglich $dir = false$ gesetzt werden. Die Implementation muss nicht geändert werden.

¹ Dieses Kapitel wurde mit Unterstützung von Gemini 3.5 Flash und 3.5 Thinking strukturell und sprachlich angepasst.

Der prinzipielle Ablauf des Bitonic Sorts lässt sich im Codebeispiel 1 nachvollziehen. Dabei handelt es sich um die lineare Implementierung. Die parallele Implementierung verändert hierbei lediglich, dass die rekursiven Aufrufe von parallelen Threads abgearbeitet werden. Auch der Name des Sortieralgorithmus wird hier ersichtlich. Der „dir“-Parameter gibt hier die Richtung an, in welche sortiert wird. Der erste Aufruf sortiert aufsteigend, der zweite absteigend. Dadurch entsteht eine bitonische Reihenfolge, welche Anschließend in `bitonicMerge(arr, low, cnt, dir);` wieder gemerged wird.

B. Bitonic Merge

Der Bitonic Merge ist der Teil des Sortieralgorithmus, in welchem Daten tatsächlich getauscht werden. Seine primäre Aufgabe ist es, eine bereits bestehende bitonische Sequenz in eine vollständig sortierte Sequenz zu überführen

Die Richtung, in welche sortiert wird, ist durch den Richtungsparameter *dir* bestimmt. Abhängig davon, wird die Sequenz monoton steigend oder monoton fallend sortiert. Auch dieser Algorithmus arbeitet nach dem „divide and conquer“-Prinzip und arbeitet iterativ durch Vergleichsoperationen sowie Swaps, bis er sich rekursiv aufspaltet [2]. Für ein zu sortierendes Array der Länge $l = 2^x, x \in N_0$, welches an Index *low* beginnt werden folgende Schritte abgearbeitet [6]:

1. Abbruchbedingung: $l \leq 1$, Ansonsten wird mit Schritt 2 fortgefahren.
2. Vergleichen: definiert wird n mit $n = \frac{l}{2}$. In folgenden Fällen werden die Elemente E_i und E_{i+n} getauscht:
 - a. $E_i > E_{i+n}$ und $dir = true$
 - b. $E_i < E_{i+n}$ und $dir = false$
Alternativ lassen sich diese Bedingungen auch als $E_i > E_{i+n} = dir$ zusammenfassen. Durch diesen Schritt werden kleine und große Elemente in die darauffolgenden Teilarrays aufgeteilt.
3. Rekursiver Aufruf: Der Algorithmus spaltet sich auf und ruft sich rekursiv für beide neuen Teilarrays der Länge k auf.

Abb. 2 zeigt diesen Ablauf der Schritte auf einem Array der Länge 16. Die Baumstruktur, welche dort deutlich wird, kann parallelisiert werden, da auf keine gemeinsamen Daten zugegriffen wird.

IV. LAUFZEITOPTIMIERUNG DURCH PARALLELISIERUNG

A. Lineare Implementierung

Der Lineare Bitonic Sort hat eine Komplexität von $O(n \log(n)^2)$. Diese entsteht zum einen durch die $\log(n)$ Rekursionsstufen des `bitonicMerge(arr, low, cnt, dir)` in welchen jeweils n Vergleiche gemacht werden.

Zum anderen ist der `bitonicSort(int[] arr, int low, int cnt, bool dir)` ebenfalls $\log(n)$ Rekursionsschritte tief, wodurch er auf eine insgesamt Komplexität von

$$O(n \log(n)) * O(\log(n)) = O(n \log(n)^2)$$

kommt.

```
private void bitonicSort(int[] arr,
    int low, int cnt, bool dir){
    if(cnt <= 1) return;
    int n = cnt / 2;
    bitonicSort(arr, low, n, true);
    bitonicSort(arr, low + n, n, false);
    bitonicMerge(arr, low, cnt, dir);
}
```

Codebeispiel 1. Implementierung der *bitonicSort* Methode eines linearen Bitonic Sorts

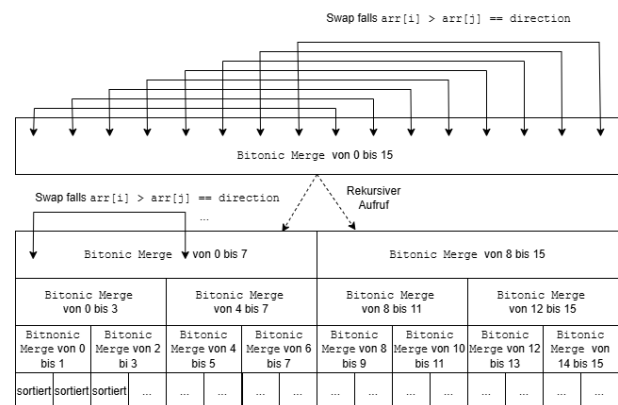


Abbildung 2 Ablauf des Bitonic Merge Prozess

B. Parallele Implementierung

Die parallele Implementierung des Bitonic Sorts hat eine Komplexität von $O(\log(n)^2)$. Prinzipiell passieren auch in diesem Algorithmus die gleichen Vergleiche, jedoch insgesamt schneller. Die Laufzeitveränderung liegt hierbei daran, dass alle Vergleiche einer Stufe gleichzeitig von Threads parallel ausgeführt werden, wodurch der Vorfaktor n wegfällt [3]. Realistisch ist dies jedoch nur umsetzbar, wenn mindestens $n/2$ Threads zur Verfügung stehen [7]. Falls der Algorithmus dementsprechend implementiert wird, ist jedoch zu bedenken, dass Threads, welche von Software erstellt werden zum einen einige Millisekunden zum initialisieren benötigen [8], (siehe Tab. 1). Des Weiteren können nicht beliebig viele Threads erzeugt werden, wodurch abgeschätzt werden muss, ab welche Arraylänge keine weiteren Threads erstellt werden sollen.

Es ist daher untypisch sowohl den parallelen als auch den linearen Bitonic Sort in Softwareanwendungen zu Implementieren. Stattdessen findet der parallele Bitonic Sort häufig Anwendung in Grafikkarten, welche Daten sortieren müssen. Dies ist auch das ursprüngliche Anwendungsgebiet, da der Bitonic Sort sich einfach und günstig in Hardware implementieren lässt. Somit hat er zwar eine höhere Komplexität als die meisten Sortieralgorithmen, hat jedoch durch seine Hardwareimplementierung im Vergleich zu softwareimplementierten Sortieralgorithmen eine kürzere Laufzeit[9].

C. Benchmarks¹

Theoretische Laufzeitkomplexität und Laufzeit in der Praxis unterscheiden sich meist durch Hardwaregegebenheiten, Overhead von Threadverwaltung und Physik [4]. Deshalb wurde die Laufzeit des linearen und

parallelen Bitonic Sorts getestet auf Arrays der Längen $l = 2^{10}$ bis $l = 2^{24}$.

Alle Berechnungen wurden auf einer Intel Core i7-10700F CPU @ 2.90GHz, mit 8 Kernen und 16 logischen Prozessoren, 16GB RAM 3200MHz und an Strom angeschlossen ausgeführt. Programmiersprache Java und Programmierumgebung Visual Studio Code. Alle Sortieralgorithmen wurden mit demselben Programm getestet. Zur Parallelisierung wurde ein ForkJoinPool genutzt mit maximal 1024 Threads. Implementierung ist agentenunterstützt entstanden.

Die Ergebnisse aus Tab. 1 zeigen typische Muster der parallelen Programmierung. So ist der parallele Bitonic Sort für kleine Arraygrößen langsamer als sein sequenzieller Gegenspieler und wird erst bei längeren Arrays effizient.

Dieses Ab einer Arraylänge von 2^{15} wird die parallele Implementierung drastisch schneller und die Laufzeit wächst langsamer an als in der linearen Implementierung. Diese steigt mit exponentiell wachsender Arraylänge ebenfalls exponentiell an.

V. VERGLEICH MIT ANDEREN SORTIERALGORITHMEN¹

Im direkten Vergleich mit etablierten, sequenziellen Sortierverfahren schneidet der lineare (sequenzielle) Bitonic Sort schlecht ab. Während klassische Algorithmen wie Merge Sort, Heap Sort und Quick Sort im durchschnittlichen Fall eine asymptotische Laufzeitkomplexität von $O(n \log(n))$ aufweisen, benötigt der sequenzielle Bitonic Sort für das Sortieren einer Sequenz eine Zeitkomplexität von $O(n \log(n)^2)$.

Dieser mathematische Nachteil spiegelt sich auch deutlich in der Praxis wider. Neben der höheren Anzahl an benötigten Operationen besitzt der Bitonic Sort eine Vorbedingung an die Arraylänge: Die Anzahl der zu sortierenden Elemente muss einer Zweierpotenz ($l = 2^x, x \in \mathbb{N}_0$) entsprechen. Ist dies nicht der Fall, muss das Array mit Füllwerten aufgefüllt werden, was die Effizienz weiter senkt. Algorithmen wie Quick Sort oder Heap Sort besitzen solche Einschränkungen nicht und agieren dementsprechend flexibel auf beliebige Datenmengen.

Die Daten aus Tab. 1 und Tab. 2 verdeutlichen diese theoretischen Schwächen: Der lineare Bitonic Sort ist zu keinem Zeitpunkt in der Lage, die Laufzeit des implementierten Quick Sorts einzuholen oder gar zu unterbieten.

Im Vergleich dazu ist die parallele Bitonic Sort deutlich besser. Da der Algorithmus auf einem datenunabhängigen Sortiernetzwerk (Sorting Network) basiert und die Reihenfolge der Vergleiche also unabhängig von den Daten im Vorfeld feststehen, lässt er sich entsprechend leicht parallelisieren [9]. Im „best case“, also unter Verwendung von n Prozessoren, sinkt die Zeitkomplexität des parallelen Bitonic Sorts auf $O(\log(n)^2)$. Dies ist zwar schneller als zuvor, ist jedoch langsamer als viele andere parallele Sortieralgorithmen wie der parallele Merge Sort.

Wie in Tab. 2 ersichtlich, ist dieser deutlich schneller als der parallele Bitonic Sort. Der Bitonic Sort bleibt also nur im GPU-Architektur-Bereich relevant, in welchem er dank Hardwarenähe stärkere Leistungen erbringen kann.

Tabelle 1 Benchmark Bitonic Sorts

Benchmark Results		
Arraylänge	Bitonic Sort (ms)	Paralleler Bitonic Sort (ms)
1024 (2^{10})	<1	6
2048 (2^{11})	<1	6
4096 (2^{12})	1	6
8192 (2^{13})	2	6
16384 (2^{14})	3	8
32768 (2^{15})	7	5
65536 (2^{16})	15	11
131072 (2^{17})	29	17
262144 (2^{18})	61	38
524288 (2^{19})	127	77
1048576 (2^{20})	264	158
2097152 (2^{21})	586	304
4194304 (2^{22})	1152	640
8388608 (2^{23})	2462	1333
16777216 (2^{24})	5085	2797

Tabelle 2 Benchmark klassische Algorithmen

Benchmark Results		
Arraylänge	Quick Sort (ms)	Paralleler Merge Sort (ms)
1024 (2^{10})	<1	<1
2048 (2^{11})	<1	<1
4096 (2^{12})	<1	<1
8192 (2^{13})	<1	<1
16384 (2^{14})	1	1
32768 (2^{15})	2	3
65536 (2^{16})	3	4
131072 (2^{17})	7	2
262144 (2^{18})	16	5
524288 (2^{19})	32	11
1048576 (2^{20})	72	18
2097152 (2^{21})	166	37
4194304 (2^{22})	465	72
8388608 (2^{23})	1427	182
16777216 (2^{24})	4889	375

VI. FAZIT

In diesem Paper wurde der parallele Bitonic Sort analysiert, seine Komplexität bestimmt und sie mit der tatsächlichen Laufzeit verglichen, welche er bereitstellt. Hierbei ist aufgefallen, dass dieser:

1. Eingeschränkten Nutzen hat durch seine Vorbedingung an die Arraylänge

2. Mit seiner Komplexität von $O(\log(n)^2)$ deutlich ineffizienter als andere parallele Algorithmen ist.

Sein Nutzen, welcher er in Hardwareimplementierung auf hochparallelisierten GPUs findet, ist dementsprechend nicht auf Softwareimplementationen für Anwendungen zu übertragen. Es empfiehlt sich, stattdessen klassische Algorithmen wie den parallelen Merge Sort oder Radix Sort zu Implementieren.

LITERATUR

- [1] K. Velusamy, T. B. Rolinger, J. McMahon, und T. A. Simon, „Exploring Parallel Bitonic Sort on a Migratory Thread Architecture“, in *2018 IEEE High Performance extreme Computing Conference (HPEC)*, 2018, S. 1–7. doi: 10.1109/HPEC.2018.8547568.
- [2] B. M. Gocal und K. E. Batchner, „Bitonic sorting on Benes/spl caron/ networks“, in *Proceedings of International Conference on Parallel Processing*, 1996, S. 749–753. doi: 10.1109/IPPS.1996.508142.
- [3] Z. Yildiz, M. Aydin, und G. Yilmaz, „Parallelization of bitonic sort and radix sort algorithms on many core GPUs“, in *2013 International Conference on Electronics, Computer and Computation (ICECCO)*, 2013, S. 326–329. doi: 10.1109/ICECCO.2013.6718294.
- [4] M. Marcellino, D. W. Pratama, S. S. Suntiarko, und K. Margi, „Comparative of Advanced Sorting Algorithms (Quick Sort, Heap Sort, Merge Sort, Intro Sort, Radix Sort) Based on Time and Memory Usage“, in *2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI)*, 2021, S. 154–160. doi: 10.1109/ICCSAI53272.2021.9609715.
- [5] NVIDIA, „Grafikkartenvergleich“, Vergleiche GeForce-Grafikkarten. Zugegriffen: 29. Mai 2026. [Online]. Verfügbar unter: <https://www.nvidia.com/de-de/geforce/graphics-cards/compare/>
- [6] Kartik, „Bitonic Sort“, Geeks For Geeks. [Online]. Verfügbar unter: <https://www.geeksforgeeks.org/dsa/bitonic-sort/>
- [7] J.-D. Lee, K.-H. Kwon, und Y.-B. Park, „Minimizing communication in bitonic sorting software“, in *Proceedings 1997 International Conference on Parallel and Distributed Systems*, 1997, S. 166–171. doi: 10.1109/ICPADS.1997.652545.
- [8] A. Castelló, R. M. Gual, S. Seo, P. Balaji, E. S. Quintana-Ortí, und A. J. Peña, „Analysis of Threading Libraries for High Performance Computing“, *IEEE Trans. Comput.*, Bd. 69, Nr. 9, S. 1279–1292, 2020, doi: 10.1109/TC.2020.2970706.
- [9] K. E. Batchner, „Sorting networks and their applications“, in *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, in AFIPS '68 (Spring). New York, NY, USA: Association for Computing Machinery, 1968, S. 307–314. doi: 10.1145/1468075.1468121.